

NAME: Vandana Illipilla

Python Built-in Functions

String Methods:

1.lower()

Converts all characters in a string to lowercase.

Ex:

```
a="HI"
```

```
a.lower()
```

Output:

```
'hi'
```

2.upper()

Converts all characters in a string to uppercase.

Ex:

```
b="python"
```

```
b.upper()
```

Output:

```
PYTHON
```

3.capitalize()

Converts the first character of the string to uppercase and the remaining characters to lowercase.

Ex:

```
c=" python programming"
```

```
c.capitalize()
```

Output:

Python programming

4.title()

Converts the first character of each word in a string to uppercase.

Ex:

```
d = "python programming"
```

```
d.title()
```

Output:

'Python Programming'

5.strip()

Removes spaces from the beginning and end of a string.

Ex:

```
e = " python "
```

```
e.strip()
```

Output:

'python'

6.replace()

Replaces a specified character or word with another character or word.

Ex:

```
f = "I like Java"
```

```
f.replace("Java", "Python")
```

Output:

'I like Python'

7.split()

Splits a string into a list of substrings.

Ex:

```
g = "Python is easy"
```

```
g.split()
```

Output:

```
['Python', 'is', 'easy']
```

8.find()

Returns the index position of the first occurrence of a specified value.

Ex:

```
h = "python"
```

```
h.find("t")
```

Output:

```
2
```

9.count()

Returns the number of times a specified value appears in a string.

Ex:

```
i = "python programming"
```

```
i.count("p")
```

Output:

```
2
```

10.startswith()

Checks whether a string starts with a specified value.

Ex:

```
j="Python"
```

```
j.startswith("Py")
```

Output:

True

11.endswith()

Checks whether a string ends with a specified value.

Ex:

```
k="Python"
```

```
k.endswith("on")
```

Output:

True

12.isdigit()

Checks whether all characters in a string are digits.

Ex:

```
l="12345"
```

```
l.isdigit()
```

Output:

True

13.isalpha()

Checks whether all characters in a string are alphabets.

Ex:

```
m = "Python"
```

```
m.isalpha()
```

Output:

True

14.join()

Combines elements of a list into a single string using a separator.

Ex:

```
n = ["Python", "AI"]
```

```
"-".join(n)
```

Output:

'Python-AI'

List([]) Methods:

A List is an ordered, mutable collection that allows duplicate values.

Ex:

```
a = ["ML", "DS", "AI", "Python"]
```

```
print(a)
```

Output:

```
["ML", "DS", "AI", "Python"]
```

List Methods

1.append()

Adds 1 item at the end

Ex:

```
a = ["ML", "DS"]
```

```
a.append("AI")
```

```
print(a)
```

Output:

```
['ML', 'DS', 'AI']
```

2.insert()

Add an element at a particular index.

```
a = ["ML", "DS"]
```

```
a.insert(1, "AI")
```

```
print(a)
```

Output:

```
['ML', 'AI', 'DS']
```

3.remove()

Delete an element by its value.

```
a = ["ML", "DS", "AI"]
```

```
a.remove("DS")
```

```
print(a)
```

Output:

```
['ML', 'AI']
```

4.pop()

Delete an element by its index.

```
a = ["ML", "DS", "AI"]
```

```
a.pop()
```

```
print(a)
```

Output:

```
['ML', 'DS']
```

5.sort()

Arrange elements alphabetically or numerically.

```
a = [30, 10, 20]
```

```
a.sort()
```

```
print(a)
```

Output:

```
[10, 20, 30]
```

6.reverse()

Display elements in reverse order.

```
a = [1, 2, 3]
```

```
a.reverse()
```

```
print(a)
```

Output:

```
[3, 2, 1]
```

7.count()

Find the number of occurrences.

```
a = [1, 2, 2, 3]
```

```
print(a.count(2))
```

Output:

```
2
```

8.index()

Find the position of an element.

```
a = ["ML", "DS", "AI"]  
print(a.index("DS"))
```

Output:

1

9.extend()

Merge two lists.

```
a = ["ML", "DS"]  
b = ["AI", "Python"]  
a.extend(b)
```

```
print(a)
```

Output:

```
['ML', 'DS', 'AI', 'Python']
```

10.clear()

Make the list empty.

```
a = ["ML", "DS", "AI"]  
a.clear()  
print(a)
```

Output:

```
[]
```

Tuple (())

A Tuple is an ordered, immutable (cannot be changed) collection that allows duplicates.

Ex

```
numbers = (10,20,30)
```

```
print(numbers)
```

Output

```
(10, 20, 30)
```

Tuple Methods

1. count()

Counts occurrences.

Ex

```
numbers = (1,2,2,3)
```

```
print(numbers.count(2))
```

Output

```
2
```

2. index()

Returns the position of an element.

Ex

```
numbers = (10,20,30)
```

```
print(numbers.index(20))
```

Output

```
1
```

Set ({})

A Set is an unordered collection of unique values (no duplicates).

Ex

```
numbers = {1,2,2,3,4}
```

```
print(numbers)
```

Output

```
{1, 2, 3, 4}
```

Set Methods

1. add()

Adds one element.

Ex:

```
numbers = {1,2}
```

```
numbers.add(3)
```

```
print(numbers)
```

Output

```
{1, 2, 3}
```

2. update()

Adds multiple elements.

Ex:

```
numbers = {1,2}
```

```
numbers.update([3,4])
```

```
print(numbers)
```

Output

{1, 2, 3, 4}

3.remove()

Removes an element.

Ex:

```
numbers = {1,2,3}
```

```
numbers.remove(2)
```

```
print(numbers)
```

Output

{1, 3}

4.discard()

Removes an element without raising an error if it doesn't exist.

Ex:

```
numbers = {1,2,3}
```

```
numbers.discard(5)
```

```
print(numbers)
```

Output

{1, 2, 3}

5.pop()

Removes a random element.

Ex:

```
numbers = {10,20,30}
```

```
numbers.pop()
```

```
print(numbers)
```

Output

{20, 30}

6.clear()

Removes all elements.

Ex:

```
numbers = {1,2,3}
```

```
numbers.clear()
```

```
print(numbers)
```

Output

set()

Dictionary ({})

A Dictionary stores data as key-value pairs. Keys are unique.

Ex:

```
student = {
```

```
    "name": "Vandana",
```

```
    "age": 22
```

```
}
```

```
print(student)
```

Output

```
{'name': 'Vandana', 'age': 22}
```

Dictionary Methods

1. keys()

Returns all keys.

Ex:

```
student = {"name": "Vandana", "age": 22}
print(student.keys())
```

Output

```
dict_keys(['name', 'age'])
```

2. values()

Returns all values.

Ex:

```
student = {"name": "Vandana", "age": 22}
print(student.values())
```

Output

```
dict_values(['Vandana', 22])
```

3. items()

Returns key-value pairs.

Ex:

```
student = {"name": "Vandana", "age": 22}
print(student.items())
```

Output

```
dict_items([('name', 'Vandana'), ('age', 22)])
```

4. get()

Gets the value of a key safely.

Ex:

```
student = {"name": "Vandana"}  
print(student.get("name"))
```

Output

Vandana

5.update()

Updates or adds key-value pairs.

Ex:

```
student = {"name": "Vandana"}  
student.update({"age": 22})  
print(student)
```

Output

```
{'name': 'Vandana', 'age': 22}
```

6.pop()

Removes a key.

Ex:

```
student = {"name": "Vandana", "age": 22}  
student.pop("age")  
print(student)
```

Output

```
{'name': 'Vandana'}
```

7.popitem()

Removes the last inserted key-value pair.

Ex:

```
student = {"name": "Vandana", "age": 22}
```

```
student.popitem()
```

```
print(student)
```

Output

```
{'name': 'Vandana'}
```

8. copy()

Creates a copy of the dictionary.

Ex:

```
student = {"name": "Vandana"}
```

```
new_student = student.copy()
```

```
print(new_student)
```

Output

```
{'name': 'Vandana'}
```

9. clear()

Removes all key-value pairs.

```
student = {"name": "Vandana", "age": 22}
```

```
student.clear()
```

```
print(student)
```

Output

```
{}
```

File Handling Methods in Python

File handling is used to create, read, write and modify files.

1. open()

Opens a file.

Syntax:

```
file = open("filename", "mode")
```

Modes:

- **r** → Read
- **w** → Write
- **a** → Append
- **x** → Create new file

Ex:

```
f = open("sample.txt", "w")
```

```
f.write("Python")
```

```
f.close()
```

Output in file:

Python

2. write()

Writes data into a file.

```
f = open("data.txt", "w")
```

```
f.write(" Python Programming")
```

```
f.close()
```

Output in file:

Python Programming

3. read()

Reads complete file content.

```
f=open("data.txt","r")
```

```
print(f.read())
```

```
f.close()
```

Output:

Python programming

4.readline()

Reads one line at a time.

File content:

Python

Java

SQL

Code:

Ex:

```
f=open("data.txt","r")
```

```
print(f.readline())
```

Output:

Python

5.readlines()

Reads all lines and returns a list.

Ex:

```
f=open("data.txt","r")
```

```
print(f.readlines())
```

Output:

```
['Python\n','Java\n','SQL']
```

6.close()

Closes the file.

Ex:

```
f.close()
```

Output:

File closed

7.tell()

Returns current file position.

```
f=open("data.txt","r")
```

```
print(f.tell())
```

Output:

0

8.seek()

Moves file pointer to a position.

Ex:

```
f=open("data.txt","r")
```

```
f.seek(5)
```

```
print(f.read())
```

Output:

Python

JSON Methods in Python

JSON = JavaScript Object Notation

Used to store and exchange data.

Python has a built-in module:

```
import json
```

1. json.dumps()

Converts Python object → JSON string

Example:

```
import json
```

```
data = {"name": "Vandana", "age": 22}
```

```
result = json.dumps(data)
```

```
print(result)
```

Output:

```
{"name": "Vandana", "age": 22}
```

2. json.loads()

Converts JSON string → Python object

Ex:

```
import json
```

```
data = '{"name": "Vandana"}'
```

```
result = json.loads(data)
```

```
print(result)
```

Output:

```
{'name': 'Vandana'}
```

3.json.dump()

Writes JSON data into a file.

Ex:

```
import json  
  
data={"name":"Python"}  
  
f=open("data.json","w")  
  
json.dump(data,f)  
  
f.close()
```

Output in file:

```
{"name":"Python"}
```

4.json.load()

Reads JSON data from file.

```
f=open("data.json","r")  
  
  
  
data=json.load(f)  
  
print(data)
```

Output:

```
{'name':'Python'}
```

Python Differences

1. find() vs index()

find()

Searches string

index()

Searches string

find()

index()

Returns -1 if not found

Gives error if not found

Ex:

```
a="python"
```

```
print(a.find("z"))
```

Output:

-1

Ex:

```
print(a.index("z"))
```

Output:

ValueError: substring not found

2. **append()** vs **extend()**

append()

Adds one element.

Ex:

```
a=[1,2]
```

```
a.append([3,4])
```

```
print(a)
```

Output:

```
[1,2,[3,4]]
```

extend()

Adds multiple elements.

Ex:

```
a=[1,2]
a.extend([3,4])
print(a)
```

Output:

```
[1,2,3,4]
```

3.remove() vspop()

remove()	pop()
-----------------	--------------

Removes value	Removes index
---------------	---------------

Returns nothing	Returns removed value
-----------------	-----------------------

Ex:

```
a=[10,20,30]
a.remove(20)
print(a)
```

Output:

```
[10,30]
```

```
a=[10,20,30]
print(a.pop(1))
```

Output:

```
20
```

4. List() vs Tuple()

List()	Tuple()
---------------	----------------

Mutable (changeable)	Immutable (cannot change)
----------------------	---------------------------

Uses []	Uses ()
---------	---------

Ex:

```
a=[1,2]
a.append(3)
print(a)
```

Output:

```
[1,2,3]
```

Ex:

```
b=(1,2)
b[0]=5
```

Output:

```
TypeError
```

5.List() vs Set()

List

Set

Allows duplicate values Does not allow duplicate values

Maintains order Order is not guaranteed

Uses [] Uses {}

Ex:

```
a=[1,2,2,3]
print(a)
```

Output:

```
[1,2,2,3]
```

Ex:

```
b={1,2,2,3}
print(b)
```

Output:

{1,2,3}

6.Dictionary vs Set

Dictionary

Stores key-value pairs

Uses key to access data

Set

Stores only values

No key concept

Ex:

```
d={"name":"Ram","age":20}
```

```
print(d)
```

Output:

```
{'name':'Ram','age':20}
```

Ex:

```
s={"Ram",20}
```

```
print(s)
```

Output:

```
{'Ram',20}
```

7.remove() vs discard() (Set)

remove()

Element not found → Error

discard()

Element not found → No error

Ex:

```
s={1,2,3}
```

```
s.remove(5)
```

Output:

KeyError

Ex:

```
s={1,2,3}
```

```
s.discard(5)
```

```
print(s)
```

Output:

```
{1,2,3}
```

8.==vs!=

=

!=

Checks values are equal Checks values are not equal

Ex:

```
a=10
```

```
b=10
```

```
print(a==b)
```

Output:

```
True
```

Ex:

```
print(a!=b)
```

Output:

```
False
```

9.==vsis

=>Checks values

```
a=[1,2]
```

```
b=[1,2]
```

```
print(a==b)
```

Output:

True

is→ Checks memory location

```
print(a is b)
```

Output:

False

10. sort() vs sorted()

sort()

Changes original list.

Ex:

```
a=[3,1,2]
```

```
a.sort()
```

```
print(a)
```

Output:

```
[1,2,3]
```

sorted()

Creates new sorted list.

Ex:

```
a=[3,1,2]
```

```
print(sorted(a))
```

Output:

```
[1,2,3]
```

11. read() vs readline() vs readlines() (File Handling)

read()

Reads the entire file content.

readline()

Reads only one line from the file.

readlines()

Reads all lines and returns them as a list.

Ex:

Python

Java

SQL

read()

f.read()

Output:

Python

Java

SQL

readline()

f.readline()

Output:

Python

readlines()

f.readlines()

Output:

['Python\n', 'Java\n', 'SQL']

12. dump() vs dumps() (JSON)

dump()

Writes JSON data to a file.

dumps()

Converts a Python object into a JSON string.

Ex:

dump()

```
import json
```

```
data = {"name": "Python"}
```

```
json.dump(data, open("a.json", "w"))
```

Output in a.json:

```
{"name": "Python"}
```

dumps()

```
import json
```

```
data = {"name": "Python"}
```

```
print(json.dumps(data))
```

Output:

```
{"name": "Python"}
```

13. load() vs loads() (JSON)

load()

Reads JSON data from a file and converts it into a Python object.

load()

```
import json
```

```
file = open("a.json", "r")
```

```
data = json.load(file)
```

```
print(data)
```

Output:

```
{'name': 'Python'}
```

loads()

loads()

Reads a JSON string and converts it into a Python object.

```
import json  
  
print(json.loads('{"a":1}'))
```

Output:

```
{'a':1}
```

14. insert() vs Slicing

insert()

Inserts a single element at a specific position.

insert()

```
a = [1,3]
```

```
a.insert(1,2)
```

```
print(a)
```

Output:

```
[1,2,3]
```

Slicing

```
a = [1,4]
```

```
a[1:1] = [2,3]
```

```
print(a)
```

Output:

```
[1,2,3,4]
```

Slicing

Can insert or replace multiple elements at once.

15. isdigit() vs isnumeric()

isdigit()

isnumeric()

isdigit()

Checks whether a string contains only digits.

isnumeric()

Checks whether a string contains any numeric characters (including fractions and other numeric symbols).

isdigit()

```
print("123".isdigit())
```

Output:

True

isnumeric()

```
print("1/2".isnumeric())
```

Output:

True

16. break vs continue

break

continue

Stops entire loop Skips current iteration

Ex:

```
for i in range(5):
```

```
    if i == 3:
```

```
        break
```

```
    print(i)
```

Output:

0

1

2

Ex:

```
for i in range(5):
```

```
    if i==3:
```

```
        continue
```

```
    print(i)
```

Output:

0

1

2

4